

Reviewer Pack — Integer Point Count in Solution Polytope Bounds Extended Frege Proof Length

Entry 4798bdb0751b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 01:54:48 UTC

Integer Point Count in Solution Polytope Bounds Extended Frege Proof Length

Entry ID: 4798bdb0751b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 01:54:48 UTC

1. Conjecture

Field A (mathematical branch): Geometry of Numbers **Field B** (complexity object): Extended Frege Proof Length

Statement:

For a 3-SAT instance Φ with n variables, the number of integer solutions $S(\Phi)$ satisfies $S(\Phi) \geq 2^{\{n - L(\Phi)\}}$, where $L(\Phi)$ is the length of the Extended Frege proof of Φ . Equality holds when Φ is a tautology with a unique solution.

Rationale (proposer's reasoning):

The geometry of numbers connects the combinatorial structure of integer solutions to algebraic constraints, while Extended Frege proofs formalize logical deductions. The conjecture posits that solution density directly impacts proof complexity via exponential trade-offs, exposing hidden geometric regularities in tautology refutations.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 087ea90c77933746

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations, product

def generate_3sat_instance(n: int) -> list:
    clauses = []
    for _ in range(2 * n):
        literals = [random.choice([1, -1]) * (i + 1) for i in random.sample(range(n), 3)]
        clauses.append(literals)
    return clauses

def dpll(sat_formula: list, assignment: dict = {}) -> bool:
    if not sat_formula:
        return True
    literal, rest = sat_formula[0]
    if abs(literal) in assignment:
        new_assignment = assignment.copy()
        new_assignment[abs(literal)] = (assignment[abs(literal)] == 1) ^ (literal < 0)
        return dpll(rest, new_assignment)
    for value in [True, False]:
        new_assignment = assignment.copy()
        new_assignment[abs(literal)] = value
        if dpll([c for c in rest if literal not in c], new_assignment):
            return True
    return False

def resolution_width(sat_formula: list) -> int:
    width = 0
    seen_literals = set()
    while sat_formula:
        clause, rest = sat_formula[0], sat_formula[1:]
        literals = [l for l in clause if abs(l) not in seen_literals]
        seen_literals.update(abs(l) for l in literals)
        new_clauses = []
        for c in rest:
            for l in literals:
                if -l in c:
                    new_clause = list(set(c) ^ {l, -l})
                    new_clauses.append(new_clause)
```

```

        break
    else:
        continue
    break
    sat_formula = new_clauses
    width = max(width, len(seen_literals))
return width

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    sat_formula = generate_3sat_instance(n)
    S_phi = sum(1 for _ in filter(dpll, [sat_formula]))
    L_phi = resolution_width(sat_formula)
    metric_value = S_phi
    conjecture_holds = S_phi >= 2**(n - L_phi)
    counterexample = "" if conjecture_holds else f"S({n})={S_phi}, L({n})={L_phi}"
    return {
        "metric_name": "Integer Point Count",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30*4 + 1, 4))
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8072c074.py", line 82, in <module>

```
result = run_trial(seed)
          ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8072c074.py", line 64, in run_trial
    S_phi = sum(1 for _ in filter(dpll, [sat_formula]))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8072c074.py", line 64, in <genexpr>
    S_phi = sum(1 for _ in filter(dpll, [sat_formula]))
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8072c074.py", line 27, in dpll
    literal, rest = sat_formula[0]
    ^^^^^^^^^^^^^^^
```

```
ValueError: too many values to unpack (expected 2)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed during execution, preventing data collection. The error suggests a structural issue with the `sat_formula` handling in the DPLL algorithm. | next: Debug the DPLL implementation's handling of `sat` formula structure to resolve the unpacking error

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	98431
2	preregistration	ollama_remote	qwen3:8b	0	0	24060
3	novelty	ollama_remote	qwen3:8b	0	0	20601
4	novelty	ollama_remote	qwen3:8b	0	0	13223
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13248
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9629
7	judge	ollama_remote	qwen3:8b	0	0	17680

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 196871 ms total latency. Provider mix: {'ollama remote': 7}

(full prompt+response transcripts available in [research/audit/4798bdb0751b.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4798bdb0751b.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4798bdb0751b.tar.gz (if generated)